

ULTRON

Advanced Personal Assistant

Master Plan & Feature Roadmap

Contents

- 00 ULTRON — Advanced Personal Assistant: Master Plan
- 01 Phase 1 — File System Access (open / find / move / rename / delete)
- 02 Phase 2 — Application Control (launch / close / list *any* app)
- 03 Phase 3 — Real WhatsApp Automation (real contacts, real send)
- 04 Phase 4 — Voice Command Engine (wider vocabulary, wake word, confirmations)
- 05 Phase 5 — Reactive Orb Animation (looks alive when you talk to it)
- 06 Phase 6 — Object Detection Upgrades
- 06b Screen Vision — "What's on my screen?"
- 07 Phase 7 — Expanded Gesture Vocabulary
- 08 Cross-Cutting: Safety & Permissions System
- 09 Roadmap & Milestones

ULTRON — Advanced Personal Assistant: Master Plan

Vision

Turn the existing ULTRON Orb UI (a voice/gesture-controlled 3D holographic front-end) into a real, always-available personal assistant for **your own machine** that can:

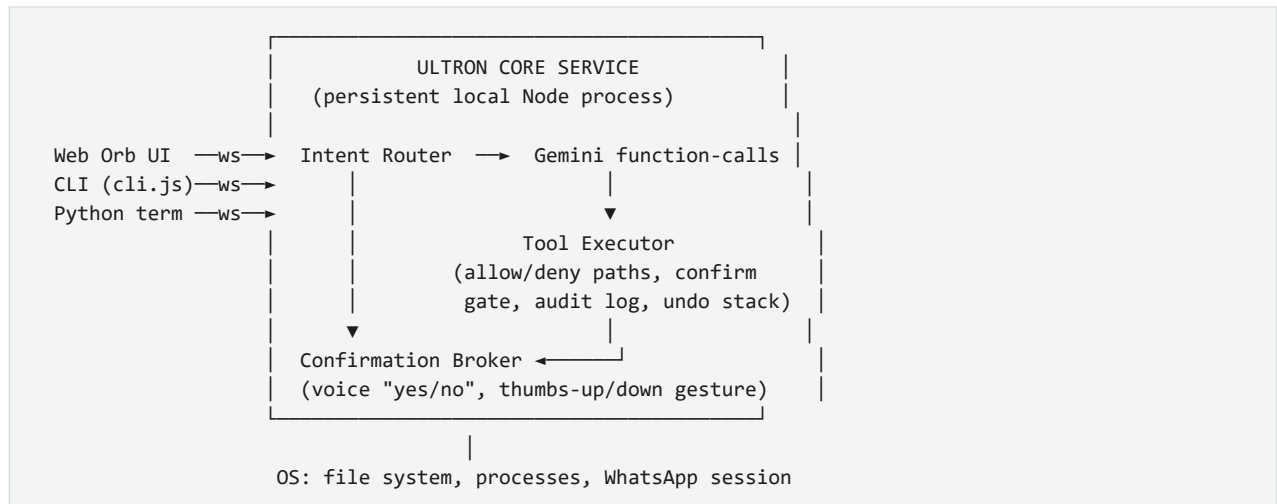
- Open, search, move, and delete files/folders by voice.
- Launch and close **any** installed application, not just a hardcoded list.
- Find a person in your real WhatsApp contacts and actually send the message (not just open a pre-filled compose box).
- React visibly and immediately to your voice — the orb should look "alive": pulsing with your speech, shifting mood for greetings/errors/success.
- Understand a much wider vocabulary of spoken commands.
- See through the camera (object detection) and understand hand gestures, including using gestures to answer yes/no prompts.
- See what's on your **screen** when you ask — "detect the screen" reads and describes whatever's currently displayed.

This is a single-user, local-machine assistant. It is **not** meant to be exposed to a network or to other users — the trust model throughout this plan assumes "whoever can talk to the microphone on this PC is authorized."

Current state (baseline, already reviewed)

- Next.js/React/Three.js orb UI, client-only.
- Voice via Web Speech API (`lib/voiceCommander.ts`), TTS also via Web Speech.
- Gestures via MediaPipe HandLandmarker (`lib/handTracker.ts`) — pinch-to-spin/zoom only.
- Object detection scaffold in `lib/objectDetector.ts`.
- Brain: `app/api/chat/route.ts` — Gemini function-calling if an API key is present, else a regex-based local intent engine. Both call the same tool executor.
- Tools today (`lib/tools/executor.ts`): open app/URL (hardcoded map), web search, recursive filename search (read-only), read file, list directory, WhatsApp deep-link (opens compose box, does not send), system telemetry, orb UI control.
- No delete/close/kill capability exists yet. No confirmation system. No audit log.
- `cli.js` and `ultron_terminal.py` duplicate the same command logic as standalone terminal front-ends.

Target architecture



Key architectural change from today: pull the tool-execution brain out of the Next.js request/response cycle and into a **persistent background core service** (Phase 5). Today everything only runs while a browser tab hits the Next dev/start server — fine for the current feature set, but it blocks two things you asked for: "always listening" wake-word behavior, and a WhatsApp session that has to stay logged in across restarts. Phases 1–4 can still be built directly against the current Next.js server; the core-service extraction is a refactor, not a rewrite, and is scheduled last (Phase 5) so we get visible features first.

Design principles used throughout this plan

- Everything destructive is reversible or confirmed.** Delete → Recycle Bin, never a permanent unlink. Close/kill app, send WhatsApp message → spoken or gestured confirmation by default (toggle-able per action type in settings).
- Every tool call is logged.** One append-only audit log so you can always see what Ultron actually did.
- Scope file/process access to what makes sense for a personal machine** — deny-list system directories (C:\Windows, Program Files, other user profiles) from delete/write operations.
- Fuzzy match voice input.** Speech-to-text is imperfect; app names, file names, and contact names should resolve via fuzzy/substring matching, not exact string equality.
- Ship incrementally.** Each phase below is independently useful and testable in the running app before moving to the next.

Phase index

Phase	Doc	Theme
1	01-file-system-access.md	Real file open/delete/move/rename with safety rails
2	02-app-control.md	Launch/close/list <i>any</i> installed app
3	03-whatsapp-automation.md	Real contact lookup + real send
4	04-voice-command-engine.md	Wake word, confirmations, wider vocabulary
5	05-reactive-animation.md	Orb reacts live to voice/mood

Phase	Doc	Theme
6	06-object-detection.md	Vision upgrades (camera)
6b	06b-screen-vision.md	"What's on my screen" — screen capture + Gemini vision
7	07-gesture-detection.md	Expanded gesture vocabulary incl. yes/no
8	08-safety-and-permissions.md	Cross-cutting safety system all phases plug into
9	09-roadmap-milestones.md	Build order, acceptance criteria, effort estimate

Read [08-safety-and-permissions.md](#) first if you only read one other doc — it's the system every risky feature (1, 2, 3) plugs into.

Phase 1 — File System Access (open / find / move / rename / delete)

Goal

Voice commands like:

- "Open my resume" → finds and opens the best-matching file with its default app.
- "Find all PDFs about taxes" → fast fuzzy search across your whole indexed area.
- "Delete that file" / "Delete report.docx" → moves it to Recycle Bin, with confirmation.
- "Close that" → closes the file/app Ultron most recently opened.
- "Rename X to Y" / "Move X to Downloads" → basic file management.

Why the current implementation isn't enough

`search_files` in `lib/tools/executor.ts` does a live recursive `fs.readdirSync` walk (depth ≤ 4) every single call. That's fine for occasional use but:

- Slow on a large home directory / won't reach deeply nested files.
- No fuzzy matching — "open my resume" won't match `Resume_2026_Final.pdf` unless the word "resume" is a substring.
- No delete/move/rename tools exist at all.
- No concept of "the file I just opened" for follow-up commands like "close that".

New tools

Add to `lib/tools/definitions.ts` + `lib/tools/executor.ts`:

Tool	Behavior
<code>open_file</code>	Fuzzy-resolves a name/description to a path (via the index below), opens with <code>Start-Process</code> (default app). Records it in the session-opened registry .
<code>close_file</code>	Closes the window most recently opened by <code>open_file</code> for that resolved path (see "closing" below). Confirmable if it's not a self-opened window.
<code>delete_file</code>	Always moves to Recycle Bin (see "soft delete" below). Never a hard unlink. Requires confirmation (Phase 8 gate) unless the user has disabled that in settings.
<code>move_file</code> / <code>rename_file</code>	Standard <code>fs.renameSync</code> , resolved against the deny-list in Phase 8.
<code>create_file</code> / <code>create_folder</code>	Simple <code>fs.writeFileSync("")</code> / <code>fs.mkdirSync</code> .
<code>search_files</code> (upgrade)	Reads from the background index instead of walking live; fuzzy-ranked results.

Background file index (replaces live directory walking)

- A lightweight indexer (Node `fs.readdir` crawl on a timer, or `chokidar` watch for live updates) builds a flat list of `{ path, name, mtime, ext }` for a configurable set of root directories (default: Desktop, Documents, Downloads, Pictures, Videos, home dir top-level — **not** the whole `C:\` drive by default, that's slow and mostly system files anyway).
- Stored as a JSON file (`data/file-index.json`) or a small SQLite DB if the list grows large (thousands of files). JSON is enough to start.
- Rebuilt on a timer (e.g. every 10 minutes) or on-demand via a "reindex my files" voice command. `chokidar` `live-watch` is a nice-to-have, not required for v1.
- Fuzzy search over the index name field (simple scoring: `exact > startsWith > substring > Levenshtein-distance-below-threshold`) so "open my resume" matches `Resume_2026_Final.pdf`, "open the tax pdf" matches `2025_Taxes.pdf`, etc.

Soft delete (Recycle Bin, never permanent)

On Windows, moving a file to the Recycle Bin (rather than `fs.unlinkSync`, which permanently deletes) is done via the Shell COM API:

```
$shell = New-Object -ComObject Shell.Application
$folder = $shell.Namespace((Split-Path $path))
$item = $folder.ParseName((Split-Path $path -Leaf))
$item.InvokeVerb("delete")
```

or more simply with the npm package `trash`, which wraps the equivalent shell call cross-platform. `delete_file` **should call this, never** `fs.unlinkSync`. This gives you a free "undo" — the file is still recoverable from the Recycle Bin even if Ultron's own undo stack (Phase 8) is somehow bypassed.

"Close that" — resolving what "that" refers to

Since the OS has no clean concept of "the file the user means", track a small **session-opened registry** in memory: `{ path, pid or window-title, openedAt }[]` for the last N things `open_file/open_app_or_file` launched. "Close that" / "close it" resolves to the most recent entry. For files opened outside of Ultron's control, closing requires matching a window by title (`Get-Process | Where MainWindowTitle -like "*name*"`) and is best-effort — document this as a known limitation, not a guarantee.

Path allow/deny list

Delete, move, rename, and create must run through the shared deny-list from [08-safety-and-permissions.md](#) — e.g. never allow deleting inside `C:\Windows`, `C:\Program Files`, another user's profile folder, or the ULTRON project directory itself. Reads (`open_file`, `search_files`) can be looser but should still skip system directories for performance/noise reasons (already partially done today via the `node_modules/AppData` skip list).

Acceptance criteria

- [] "Open my resume" opens the right file within the indexed directories.
- [] "Delete report.docx" asks for confirmation, then the file lands in Recycle Bin (verifiable by restoring it from there).
- [] Attempting to delete something inside a denied path is refused with a clear spoken explanation, not a silent failure.
- [] "Close that" closes the file/app most recently opened by Ultron in the same session.
- [] File search returns results in well under a second on a typical Documents/Desktop/Downloads-sized index.

Phase 2 — Application Control (launch / close / list *any* app)

Goal

"Open Spotify", "open [any app you have installed, not just the ~20 in the current hardcoded map]", "close Chrome", "what apps are running", "switch to Discord".

Why the current implementation isn't enough

APP_LAUNCH_MAP in lib/tools/executor.ts (and duplicated in cli.js and ultron_terminal.py) is a fixed dictionary of ~30 apps. Anything not in that list falls through to launchTarget(rawTarget), which only works if the raw target happens to be a valid executable name/URL already on PATH — no real "any app" support, and there's no close/kill/list capability at all.

Dynamic app discovery

Replace the fixed map with a **discovered app catalog**, built once at startup and refreshed periodically:

```
Get-StartApps | Select-Object Name, AppID
```

This returns every Start Menu entry (both classic .exe/.lnk shortcuts and UWP/Store apps) with a launchable AppID — explorer.exe shell:AppsFolder\<AppID> launches any of them uniformly. Merge this with a small manual alias table for the handful of apps whose voice-friendly name differs a lot from their Start Menu name (kept from today's APP_LAUNCH_MAP, but now used purely as an alias layer on top of discovery — not as the only source of truth).

Cache the catalog to data/app-catalog.json, refresh in the background (e.g. on Ultron startup + every hour), so lookups are instant.

Fuzzy resolution

Voice transcription of app names is noisy ("open v.s. code", "open discourse" for Discord). Resolve the spoken target against the app catalog using the same fuzzy scoring approach as file search (exact > startsWith > substring > edit-distance), and fall back to the manual alias table, then to the raw Start-Process attempt as today's last resort.

New tools

	Behavior	
lled_apps	Returns the discovered catalog (useful for "what apps do I have").	
or_file (upgrade)	Resolves against catalog + aliases + fuzzy match, records into the session-opened registry (shared with Phase 1).	

	Behavior	
ing_apps	`Get-Process \	Where MainWindowTitle -ne "" → friendly list of open windows/apps.
	Finds matching running process by name or window title, sends a graceful close (WM_CLOSE via taskkill /PID <pid> without /F first), falls back to /F (force) only after a short timeout if it didn't close — reduces risk of killing an app mid-unsaved-work without at least trying the graceful path.	
_app	Brings an already-running app's window to the foreground (e.g. via a small PowerShell Add-Type + SetForegroundWindow call, or the nircmd/AutoHotkey-style approach if we want to avoid inline C# — pick one implementation and keep it in one helper).	

Confirmation

close_app (especially the forced/taskkill /F path) is a destructive-ish action — closing something with unsaved work loses that work. Route it through the Phase 8 confirmation gate by default ("Close Notepad? It might have unsaved changes." → confirm), configurable off for apps the user marks as "always safe to close" (e.g. browsers, media players) in settings.

Acceptance criteria

- [] "Open [any app in your Start Menu, not just the current hardcoded 30]" launches it correctly.
- [] "What's running right now" lists actual open windows.
- [] "Close Spotify" gracefully closes it (or asks to confirm/force if it doesn't respond).
- [] Misheard app names ("open v s code") still resolve correctly via fuzzy matching against the real catalog.

Phase 3 — Real WhatsApp Automation (real contacts, real send)

Goal

"Message my brother saying I'm on my way" → Ultron finds **your brother** in your **real WhatsApp contact list** (not the placeholder `data/contacts.json`) and the message is **actually sent**, not just pre-filled waiting for a manual click.

Why the current implementation isn't enough

`send_whatsapp_message` opens `web.whatsapp.com/send?phone=...&text=...`, which pre-fills the compose box but requires you to click Send yourself. Contact resolution is against a hand-maintained `data/contacts.json` (currently just placeholder entries with empty phone numbers) — it has no idea who your actual WhatsApp contacts are.

Approach: `whatsapp-web.js`

Use the `whatsapp-web.js` Node library, which drives a real WhatsApp Web session via a background headless/headed Chromium instance (puppeteer under the hood). One-time setup:

- 1 First run shows a QR code (in terminal or a small UI panel) — scan it once with your phone's WhatsApp, same as logging into WhatsApp Web normally.
- 2 The library persists the authenticated session locally (`LocalAuth` strategy writes session data to disk), so you don't re-scan on every restart.
- 3 This becomes a background connection owned by the **core service** (see Phase 5) rather than something spun up per-request, since keeping the session alive is the whole point.

Once connected, it gives you:

- `client.getContacts()` — your real contact list (name + number), replacing `data/contacts.json` as a live cache instead of a hand-edited placeholder.
- `client.sendMessage(chatId, text)` — an actual send, no manual click.
- (Optional, later) `client.on('message', ...)` — could support "read me my last message from mom", if you want that eventually; not required for v1.

New tools

Tool	Behavior
<code>whatsapp_list_contacts</code>	Returns/refreshes the cached real contact list.
<code>whatsapp_send_message</code> (replaces current)	Fuzzy-resolves recipient against real contacts (name, saved relationship label if you keep a small local nickname map like "brother" → contact name, or phone digits), then calls <code>sendMessage</code> directly.

Contact resolution priority

- 1 Exact/fuzzy match against real WhatsApp contact display names.
- 2 A small local **nickname map** (kept in `data/contact-aliases.json`) so "my brother" / "mom" continue to work — this file just maps a nickname to the *real* contact's phone number/WhatsApp ID, rather than storing the number itself as a fake placeholder like today.
- 3 A raw phone number spoken directly.

Confirmation before send

Because this now actually sends (no manual click as a safety net anymore), route every `whatsapp_send_message` call through a spoken confirmation by default: *"Sending to Rohan: 'On my way.' Confirm?"* — this can be turned off in settings for a faster, fully hands-free flow once you trust the contact resolution, but it should default ON given a misheard name or number would otherwise silently message the wrong person.

Known risk — please read

`whatsapp-web.js` (and any unofficial WhatsApp Web automation library) is not an officially sanctioned integration; WhatsApp's terms of service technically prohibit unofficial clients/automation, and there is a **non-zero risk of your account being flagged or temporarily restricted**, especially with high message volume or bulk/spam-like patterns. For personal, low-volume, 1:1 messaging triggered by you speaking to your own assistant, real-world risk is low, but it's not zero, and it's worth deciding you're comfortable with that trade-off before this phase is built. If you'd rather avoid that risk entirely, the fallback is to keep today's deep-link-plus-manual-click behavior (Phase 3 would then be skipped, and only contact resolution — matching against a manually maintained richer contact list — gets improved).

Acceptance criteria

- [] First-run QR pairing works and persists across restarts (no repeat scans).
- [] "Message my brother saying I'll be late" resolves the correct real contact and actually delivers the message after confirmation.
- [] A misheard/unknown name triggers a spoken "I couldn't find X in your contacts" rather than silently messaging the wrong person.

Phase 4 — Voice Command Engine (wider vocabulary, wake word, confirmations)

Goal

Understand "every possible command" as broadly as practical, activate without pressing a key, and support yes/no confirmation for the risky actions introduced in Phases 1–3.

What already works well

`app/api/chat/route.ts`'s Gemini function-calling path is already a good, general NLU layer — it maps free-form speech to the tool schema in `lib/tools/definitions.ts` without needing per-phrase regexes. As we add tools (Phases 1–3), this path scales automatically: **expanding** `ASSISTANT_TOOLS` **is most of "understanding more commands."** The regex-based `executeLocalIntentEngine` fallback (for when there's no Gemini key) is necessarily narrower and will need matching updates for the highest-value new commands (open/delete file, close app), but doesn't need to cover everything — it's explicitly the "no API key" fallback, not the primary brain.

Wake word ("Hey Ultron")

Currently voice listening is toggled by pressing `v` or clicking a button. For a more assistant-like feel, add always-on wake-word detection:

- Use [Porcupine](#) (on-device, works offline, has a free tier for personal use) with a custom "Hey Ultron" keyword, or a simpler open-source keyword spotter if avoiding a third-party SDK is preferred.
- Wake-word listener runs continuously at low resource cost; on detection, it triggers full `VoiceCommander` speech recognition for the actual command, same as pressing `v` today.
- This only makes full sense once Phase 5's background core service exists — a browser tab has to stay open and focused for continuous mic access otherwise. Until then, ship this phase with manual activation retained and wake-word as a stretch add-on if the browser-tab constraint is acceptable to you.

Confirmation subsystem

A cross-cutting addition used by Phases 1–3: any tool definition gains a `requiresConfirmation: boolean` (default true for `delete_file`, `close_app`, `whatsapp_send_message`; default false for read-only tools). Flow:

- 1 Gemini (or the local engine) decides to call a guarded tool.
- 2 Instead of executing immediately, the orchestrator emits a spoken/UI prompt: *"Delete report.docx — are you sure?"*
- 3 User responds "yes"/"confirm"/"do it" (voice) or a thumbs-up gesture (Phase 7), or "no"/"cancel"/thumbs-down to abort.
- 4 Only on affirmative confirmation does `executeTool` actually run.

This needs a small piece of conversation state (pending action + timeout, e.g. auto-cancel after 15s of silence) added to `JarvisOrb.tsx`'s command handling loop.

Settings for confirmation policy

A simple `data/settings.json` (or extend the existing `localStorage`-based config panel already visible in `JarvisOrb.tsx`) lets you toggle confirmation requirements per action type, e.g. "always confirm deletes" (recommended, on by default) vs "never confirm app closes" (fine to turn off since Phase 2 already tries a graceful close first).

Acceptance criteria

- [] Every new tool from Phases 1–3 is reachable via natural free-form speech through the Gemini path, no exact-phrase matching required.
- [] A guarded action (delete/close/send) always pauses for confirmation unless explicitly disabled in settings for that action type.
- [] Saying "no"/"cancel" during a confirmation prompt cleanly aborts with no side effects.

Phase 5 — Reactive Orb Animation (looks alive when you talk to it)

Goal

When you speak, the orb visibly reacts in real time — not just a static listening indicator. Saying "hello" gets a distinct friendly animation; errors look different from success; the orb pulses in sync with your voice and with its own spoken replies.

What already exists to build on

`lib/voiceCommander.ts` already sets up a `Web Audio AnalyserNode` on the microphone stream for the equalizer wave visualizer, and `lib/orbScene.ts` exposes an `OrbSceneApi` with theme/pulse control. The wiring between "live audio amplitude" and "core geometry" isn't fully connected yet — the equalizer bars react, but the orb's core shells/emissive intensity don't.

Emotion state machine

Add `setEmotionState(state: EmotionState)` to `OrbSceneApi` in `lib/orbScene.ts`, with states:

State	Trigger	Visual treatment
idle	Default, nothing happening	Slow ambient rotation, baseline glow
listening	Mic active, waiting for speech	Core pulses with live mic amplitude (see below), outer ring brightens
thinking	Waiting on Gemini API response	Faster inward-spiraling particle motion, subtle color desaturation
speaking	TTS is playing the response	Core pulses with TTS output amplitude (a "talking" look)
executing	A tool call is running (opening app, deleting file, etc.)	Quick outward pulse burst + scan-ring sweep
greeting	Wake word / "hello" detected	A one-off bright, friendly full-color pulse + brief rotation flourish, distinct from overdrive
success	Tool call completed OK	Short green-tinted ring flash (independent of active theme color) regardless of current protocol, so success always reads clearly
error	Tool call failed / API error	Short red flicker + slight shake/jitter on the core
confirm_pending	Waiting on a yes/no confirmation (Phase 4)	Slow amber double-pulse "waiting for input" look, distinct from <code>listening</code>

Each state is a small, self-contained animation preset (color tint, pulse frequency/amplitude, particle behavior) layered on top of the existing theme system (crimson/jarvis/gold/etc.) — theme picks the base palette, emotion state picks the transient behavior on top of it.

Audio-amplitude-driven core pulse

Feed the existing `AnalyserNode` amplitude data (already computed for the equalizer) into the orb core's scale/emissive-intensity each frame while in `listening` state, and do the same using an analyser on the **TTS output** audio while in `speaking` state (Web Speech API TTS doesn't expose raw audio directly, so approximate this with an envelope generator synced to utterance start/duration, or route TTS through the Web Audio API via `speechSynthesis` + a `MediaElementAudioSourceNode` where the browser allows it — document that a real amplitude-accurate lip-sync isn't fully achievable with Web Speech TTS everywhere, and an approximated envelope is an acceptable v1).

Acceptance criteria

- [] Speaking louder/softer while Ultron is listening visibly changes the orb's pulse in real time (not just the equalizer bars).
- [] Saying "hello" produces a distinct, recognizable greeting animation vs. idle state.
- [] A failed action visibly looks different (red flicker) from a successful one (green flash) without checking the text log.
- [] A pending confirmation (Phase 4) has its own clearly distinct look so you know Ultron is waiting on you specifically.

Phase 6 — Object Detection Upgrades

Goal

Point the camera at something and ask "what is this" for a real answer, with visible on-screen recognition rather than a background-only feed.

Current state

`lib/objectDetector.ts`'s `CameraObjectDetector` already runs live detection and feeds labels into `app/api/chat/route.ts` as `detectedObjects`, which Gemini receives as context text alongside the camera frame (imageBase64 multimodal input already wired up). This is a solid foundation — this phase is about tightening the loop, not building from scratch.

Upgrades

- 1 **On-screen bounding boxes / labels** — currently detection presumably feeds labels into state without a visible overlay (verify against current component during implementation); add a canvas overlay drawing boxes + confidence-labeled tags over the live camera feed so you can see what Ultron sees.
- 2 **"What is this" snapshot flow** — a dedicated `capture_and_identify` tool: grabs one frame at the moment you ask, sends it to Gemini multimodal with the current `detectedObjects` as a hint, and reads back a specific answer ("That looks like a Nintendo Switch controller") instead of a generic "I have optical tracking active" filler currently returned when `detectedObjects` is empty.
- 3 **Continuous vs. on-demand modes** — keep continuous light-weight detection for the HUD label feed, but only send a frame to Gemini (network call) on explicit request, to avoid constant API usage/latency.
- 4 **Model check** — confirm which model `CameraObjectDetector` currently loads (`MediaPipe ObjectDetector` task vs. something else) and, if it's a small default model, evaluate swapping to `EfficientDet-Lite2` for better accuracy at an acceptable perf cost on your hardware — this is a one-line model asset swap once confirmed, not a rewrite.

Acceptance criteria

- [] Detected objects are visibly boxed/labeled on the live feed, not just logged internally.
- [] "What am I holding" / "what is this" gives a specific, useful answer tied to what's actually in frame at that moment.

Screen Vision — "What's on my screen?"

Goal

"Detect the screen" / "what's on my screen" / "read my screen" / "what am I looking at" → Ultron captures what's currently displayed and gives you a spoken description or answers a specific question about it — "summarize this article", "what does this error say", "what's this code doing".

Why this is separate from camera object detection

Camera vision ([06-object-detection.md](#)) sees the physical world through the webcam. Screen vision sees the desktop itself — browser tabs, open documents, code editors, error dialogs. Different capture source, same downstream flow: grab one frame, send it to Gemini multimodal, speak back the answer — this reuses the exact `imageBase64` path already wired up in `app/api/chat/route.ts` for camera vision.

Recommended approach: browser Screen Capture API, not a silent native screenshot

There are two ways to grab "what's on the screen," and the choice here matters more than it looks:

A) Browser `getDisplayMedia()` — recommended. `navigator.mediaDevices.getDisplayMedia()` triggers the OS's own native "choose what to share" picker — the same dialog you'd see sharing your screen on a video call (pick a specific window, an app, or the whole screen). Grab a single frame from the returned stream onto an off-screen `<canvas>`, export it as a JPEG data URL, send it to `/api/chat` as `imageBase64`. Stop the stream immediately after capturing that one frame — don't leave a live screen-recording session running — unless you deliberately turn on a continuous "watch my screen" mode.

B) Native OS screenshot (e.g. a `PowerShell System.Drawing / Windows.Graphics.Capture` call triggered directly by the tool executor, the same pattern `open_app_or_file` already uses to shell out) — technically simpler, no browser API needed, **but has no visible consent step**. A spoken phrase would silently capture literally anything on screen at that instant — a password manager left unlocked in another window, a private chat, a banking dashboard — with zero on-screen indication it happened. That's a meaningfully bigger privacy exposure than anything else in this plan, bigger than file delete, bigger than camera vision (a webcam at least has a hardware light; an instant background screenshot has no equivalent tell).

Recommendation: build option A only. Option B is documented here so the trade-off is explicit, not because it's the plan — don't build the silent native path unless you've deliberately decided you want it despite the above.

Flow (option A)

- 1 You say "detect the screen" / "what's on my screen".
- 2 If no screen-share session is active yet this session, the browser's native picker opens (you choose a window/app/whole screen) — this happens once per browser session, not on every single request, unless you revoke sharing in between.
- 3 Ultron grabs one frame from the shared stream and immediately stops the stream (default one-shot mode) — or keeps it alive only if you've explicitly enabled continuous "watch my screen" mode.

- 4 The frame is sent to Gemini multimodal alongside your spoken question/instruction, exactly like the existing camera vision flow.
- 5 The answer comes back and is spoken, same as any other response.

New tool

Tool	Behavior
read_screen	Captures one frame via <code>getDisplayMedia()</code> and calls the existing chat endpoint with <code>imageBase64</code> + your question.

Because the capture step has to happen in the browser — the Node process running the Next.js server has no access to your monitor's framebuffer on its own — `read_screen` is best modeled as **client-initiated**: when Gemini's function-calling decides to call `read_screen` with no `imageBase64` attached yet, `components/JarvisOrb.tsx` intercepts that specific tool call, performs the capture, and re-sends the message with the frame attached, rather than the server trying to execute it the way the other OS-level tools in `lib/tools/executor.ts` run headlessly.

Privacy notes

- Every screen read sends that frame to Google's Gemini API — same as the existing camera "what is this" flow, but screen content is more likely to contain sensitive material (open emails, financial dashboards, private messages) than a webcam frame. A one-time spoken/UI notice the first time it's used per session ("Screen captures are sent to Gemini for analysis — go ahead?") is a better fit than requiring confirmation on every single read, which would get old fast for something you'd likely use often. This is a one-time acknowledgment, distinct from the per-action confirmation gate in [08-safety-and-permissions.md](#), which is reserved for destructive actions.
- Never auto-select "entire screen" as a default — let the browser's own picker apply its default, and log every screen read in the same audit log as every other tool call ([08-safety-and-permissions.md](#)).
- One-shot capture is the default. A "watch my screen continuously" mode is a bigger step — effectively a standing screen-share — and should be opt-in, with the orb showing a distinct, persistent "screen-watching" visual state (per the emotion-state system in [05-reactive-animation.md](#)) for the entire time it's active, not just at the moment you trigger it.

Acceptance criteria

- [] "What's on my screen" opens the native share picker once, captures a frame, and returns an accurate spoken description of the actual screen contents.
- [] "What does this error say" (with an error dialog visible) reads out the real error text, not a generic description.
- [] No screenshot is ever taken without the OS-level picker appearing at least once per session — no silent background captures.
- [] The screen-share stream stops immediately after each one-shot capture (verify the browser's tab/toolbar "sharing" indicator disappears) unless continuous mode is explicitly enabled.

Phase 7 — Expanded Gesture Vocabulary

Goal

Beyond today's pinch-to-spin/zoom, add a small set of high-value gestures, most importantly **hands-free yes/no** so the confirmation flow from Phase 4 doesn't always require speaking.

Current state

`lib/handTracker.ts`'s `HandTracker` already tracks landmarks with pinch hysteresis for one-hand spin and two-hand zoom (`GestureMode = "idle" | "spin" | "zoom"`). The landmark pipeline (`MediaPipe HandLandmarker`) already gives us everything needed for more gesture classification — this is additive recognition logic on data already being computed, not a new tracking system.

New gestures

Gesture	Recognition	Action
Thumbs-up	Thumb tip extended away from a closed fist, pointing up	Confirm ("yes") for a pending confirmation prompt (Phase 4)
Thumbs-down	Same shape, pointing down	Deny ("no") for a pending confirmation prompt
Open palm hold (~1s)	All 5 fingertips extended and roughly stationary	Toggle mute, matching the <code>M</code> key today
Closed fist hold (~1s)	All fingertips curled near palm	"Stop" — halts auto-spin / cancels an in-progress confirmation, matching "freeze"/"halt" voice commands
Two-hand twist	Both hands pinching, rotating relative to each other	Fine rotation control (supplements today's zoom-by-spreading)

Each gesture needs a short hold/hysteresis window (similar to the existing pinch hysteresis) to avoid false triggers during natural hand movement.

Wiring into the confirmation system

`HandTrackerCallbacks` gains `onConfirm()` / `onDeny()` callbacks, which `JarvisOrb.tsx` treats identically to the voice "yes"/"no" path from Phase 4 — one confirmation resolver, two possible input methods (voice or gesture).

Acceptance criteria

- [] A thumbs-up/down gesture resolves a pending confirmation exactly like saying "yes"/"no" would.
- [] New gestures don't cause false positives during normal spin/zoom use (tested by deliberately doing rapid spin/zoom motions and confirming no accidental confirm/deny/mute triggers).

Cross-Cutting: Safety & Permissions System

Every risky feature in Phases 1–3 plugs into this one system, rather than each implementing its own ad hoc safety logic. Build this alongside Phase 1 (file access is the first place it's needed).

Why this matters even for a personal single-user assistant

You asked for the assistant to "execute every possible task" and to be able to delete files — that's a reasonable thing to want for your own machine, but a voice/LLM-driven pipeline has two failure modes worth designing around from day one rather than retrofitting later:

- 1 **Speech-to-text or LLM misinterpretation** — "delete the report" when you meant a different file with a similar name, or a misheard app name closing the wrong window.
- 2 **No manual "click Send" safety net anymore** once Phase 3 lands — today's WhatsApp deep-link accidentally already protects you by requiring a manual click; real automation removes that, so something has to replace it.

None of this is about distrust of you — it's about the same class of mistake a human assistant would also want a "wait, confirm that?" moment for before deleting something or messaging someone.

The four pillars

1. Soft-delete only

No tool ever calls `fs.unlinkSync` or an equivalent permanent delete. Every delete goes through the OS Recycle Bin (trash npm package or Shell COM call — see [01-file-system-access.md](#)). This alone makes almost any delete mistake fully recoverable in seconds.

2. Confirmation gate

A shared `requiresConfirmation` flag per tool (see [04-voice-command-engine.md](#)), defaulting **on** for: `delete_file`, `close_app` (forced/kill path), `whatsapp_send_message`, `move_file/rename_file` when the destination overwrites an existing file. Defaulting **off** for anything read-only (`search_files`, `read_file`, `list_directory`, `get_installed_apps`, `list_running_apps`, `get_system_telemetry`) and for `open_file/open_app_or_file` (opening things is low-risk and reversible — just close it again).

You can turn confirmation off per action type in settings once you trust the system, but the plan ships with it on by default for the four listed above.

3. Path & process deny-list

A single shared config (`data/safety-config.json`) listing directories that `delete_file/move_file/rename_file/create_file` refuse to touch, e.g.:

```
{
  "deniedPaths": [
    "C:\\Windows",
    "C:\\Program Files",
    "C:\\Program Files (x86)",
    "C:\\Users\\<you>\\AppData",
    "<this project's own directory>"
  ],
  "protectedProcesses": ["explorer.exe", "csrss.exe", "winlogon.exe", "services.exe"]
}
```

Checked in the executor before any destructive filesystem or process action runs — a request that resolves inside a denied path is refused with a clear spoken reason ("I won't delete anything inside Windows system folders"), not a silent no-op.

4. Audit log

Every tool call — guarded or not — appends one line to `data/audit-log.jsonl`:

```
{"ts":"2026-08-27T10:15:00Z","tool":"delete_file","args":{"path":"C:\\Users\\...\\report.docx"},"confirmed":true,"result":
```

This is your single source of truth for "what did Ultron actually do" — cheap to build (one `fs.appendFileSync` call in the executor's wrapper), and valuable the first time something unexpected happens and you want to know why.

Optional: kill switch

A single command — voice ("Ultron, stand down") or a UI button — that flips a runtime flag disabling all guarded/destructive tools immediately, leaving only read-only tools (search, open, telemetry) active until re-enabled. Cheap insurance, not required for v1 but worth having before Phase 3 ships.

What this plan deliberately does NOT restrict

Reading files, opening apps/files, web search, telemetry, and object/gesture detection all stay frictionless with no confirmation — the safety system is scoped specifically to actions that are hard to undo (permanent-feeling delete, force-closing an app with unsaved work, sending a message to another person), matching the same "reversible vs. hard-to-reverse" distinction used for any assistant acting on your behalf.

Roadmap & Milestones

Build order is chosen so every phase ships something you can actually use before starting the next, and so the safety system (Phase 8) is in place *before* the first destructive tool (delete) goes live — not bolted on after.

Milestone 1 — Safe file & app control (Phases 8 + 1 + 2)

The biggest capability jump with the least new external dependency risk (no third-party services, everything is local OS calls).

- Safety system: soft-delete helper, confirmation gate, deny-list config, audit log.
- File tools: `open_file` (fuzzy, indexed), `search_files` (upgraded), `delete_file` (Recycle Bin + confirm), `move_file`, `rename_file`, `create_file/create_folder`, `close_file` (best-effort via session-opened registry).
- App tools: dynamic catalog via Get-StartApps, fuzzy `open_app_or_file`, `list_running_apps`, `close_app` (graceful → forced + confirm), `switch_to_app`.
- Voice engine: expand `ASSISTANT_TOOLS`, wire confirmation prompts into `JarvisOrb.tsx`'s command loop (voice yes/no only — gesture yes/no comes in Milestone 3).

Done when: you can say "open my resume", "delete that old screenshot", "close Discord", "what's running", and each works correctly with a delete/close confirmation you can accept or reject by voice.

Milestone 2 — Real WhatsApp (Phase 3)

Gated on you being comfortable with the ToS/account-risk trade-off described in [03-whatsapp-automation.md](#).

- `whatsapp-web.js` integration + one-time QR pairing.
- Real contact list replacing the placeholder `data/contacts.json`, plus a small nickname-alias file for "brother"/"mom"-style commands.
- `whatsapp_send_message` now actually sends, gated by confirmation.

Done when: "message my brother saying I'm on my way" finds your real brother in WhatsApp and delivers the message after a confirm.

Milestone 3 — The orb feels alive (Phases 5 + 7)

Pure front-end work, no new external services — good to interleave with Milestone 1/2 if you want visible progress sooner since it doesn't depend on them.

- Emotion state machine in `orbScene.ts` (idle/listening/thinking/speaking/executing/greeting/success/error/confirm_pending).
- Live mic-amplitude-driven core pulse while listening; TTS-envelope-driven pulse while speaking.
- Gesture additions: thumbs-up/down wired to the confirmation resolver from Milestone 1, open-palm-hold mute, fist-hold stop.

Done when: the orb visibly and distinctly reacts to greetings, errors, success, and a pending confirmation — and you can thumbs-up/down a confirmation instead of speaking.

Milestone 4 — Vision upgrade (Phase 6 + Screen Vision)

- On-screen bounding boxes for detected objects.
- `capture_and_identify` tool for a specific "what is this" answer.
- Model upgrade if warranted after checking the current model in `lib/objectDetector.ts`.
- `read_screen` tool: `getDisplayMedia()`-based one-shot screen capture routed through the same Gemini multimodal path, per [06b-screen-vision.md](#) — including the one-time "screen captures are sent to Gemini" notice and the visible "screen-watching" orb state if continuous mode is ever added.

Done when: pointing the camera at something and asking "what is this" gives a specific, accurate spoken answer with a visible box around the object, **and** saying "what's on my screen" opens the native share picker once, captures a frame, and gives an accurate spoken description of what's actually displayed.

Milestone 5 — Always-on core service (Phase 5's architectural piece)

The bigger refactor: extract the tool executor + Gemini orchestration + the WhatsApp session out of the per-request Next.js API route into a persistent background Node process that the web orb, `cli.js`, and `ultron_terminal.py` all connect to (e.g. over a local WebSocket), plus wake-word listening ("Hey Ultron") that doesn't require a focused browser tab.

This is scheduled last deliberately — it's a structural change, highest effort, and everything in Milestones 1–4 works and is testable without it (just with the current constraint that the browser tab needs to be open/focused, and the WhatsApp session reconnects per Next.js server restart rather than truly persisting).

Done when: you can say "Hey Ultron, open Spotify" without a browser tab focused or a key pressed, and it works.

Suggested first step

Start Milestone 1 with just Phase 8 (safety system) + `delete_file` + `open_file` end-to-end, as the smallest slice that proves the whole confirm → execute → audit-log loop works, before building out the rest of the file/app tool surface on top of a pattern you've already seen work.